



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ТЕОРИЯ ИГР И ИССЛЕДОВАНИЕ ОПЕРАЦИЙ

Лабораторная работа № 9

на тему:

«Игры на разорение. Нахождение цены игры и оптимальных
смешанных стратегий»

Вариант 13

Студент:

ИУ8-105
(Группа)

Б. Е. Шекунов
(И. О. Фамилия)

Преподаватель:

Н. С. Коннова
(И. О. Фамилия)

Москва, 2026 г.

Цель работы:

Изучить игры на разорение, то есть игры с ограниченными ресурсами, и найти цену игры и оптимальные стратегии игроков. В работе рассматриваются два типа задач: игра «инспектирование» и игра «женщины и кошки против мышей и мужчин».

Постановка задачи:

Для игры на разорение требуется вычислить цену игры и оптимальные смешанные стратегии игроков.

Для примера «инспектирование» задана рекурсия:

$$v_1 = 0,$$
$$v_N = \frac{v_{N-1} + 1}{-v_{N-1} + 3}.$$

Также используется замкнутая формула:

$$v_N = \frac{N - 1}{N + 1}.$$

Оптимальные стратегии игроков имеют вид:

$$x^{(N)} = y^{(N)} = \left(\frac{1}{N + 1}, \frac{N}{N + 1} \right).$$

Для второй задачи рассматривается игра между двумя группами:

$$v(m_1, m_2; n_1, n_2),$$

где:

m_1 — число женщин,

m_2 — число кошек,

n_1 — число мышей,

n_2 — число мужчин.

Цена игры находится рекурсивно через значения подыгр.

Исходные данные:

Вариант для игры «инспектирование»

$$N = 13.$$

Вариант для игры «женщины и кошки против мышей и мужчин»

$$m_1 = 3, m_2 = 4, n_1 = 2, n_2 = 3.$$

То есть необходимо найти:

$$v(3,4; 2,3).$$

Ход работы:

Вариант «инспектирование», $N = 13$

Последовательно по рекурсии получено:

$$v_{12} = \frac{11}{13}.$$

Тогда:

$$v_{13} = \frac{v_{12} + 1}{-v_{12} + 3}.$$

Подставим значение v_{12} :

$$v_{13} = \frac{\frac{11}{13} + 1}{-\frac{11}{13} + 3}.$$
$$v_{13} = \frac{24}{28} = \frac{6}{7}.$$

Проверка по замкнутой формуле:

$$v_{13} = \frac{13 - 1}{13 + 1} = \frac{12}{14} = \frac{6}{7}.$$

Следовательно, цена игры:

$$v_{13} = \frac{6}{7} \approx 0.857143.$$

Эффективная матрица игры:

$$G_{13} = \begin{pmatrix} -1 & 1 \\ 1 & \frac{11}{13} \end{pmatrix}.$$

Нижняя цена игры:

$$\underline{v} = \frac{11}{13} \approx 0.846154.$$

Верхняя цена игры:

$$\bar{v} = 1.$$

Так как:

$$\underline{v} < \overline{v},$$

седловой точки в чистых стратегиях нет, поэтому решение ищется в смешанных стратегиях.

Оптимальная стратегия первого игрока:

$$x^{(13)} = \left(\frac{1}{14}, \frac{13}{14} \right).$$

Оптимальная стратегия второго игрока:

$$y^{(13)} = \left(\frac{1}{14}, \frac{13}{14} \right).$$

В десятичном виде:

$$x^{(13)} = y^{(13)} \approx (0.071429, 0.928571).$$

Вариант «женщины и кошки против мышей и мужчин»

$$m_1 = 3, m_2 = 4, n_1 = 2, n_2 = 3.$$

Необходимо найти:

$$v(3,4; 2,3).$$

Для построения матрицы очередного хода были вычислены значения подыгр:

$$v(m_2 - 1) = v(3,3; 2,3) = \frac{2}{7} \approx 0.285714,$$

$$v(n_1 - 1) = v(3,4; 1,3) = \frac{16}{21} \approx 0.761905,$$

$$v(n_2 - 1) = v(3,4; 2,2) = \frac{2}{3} \approx 0.666667,$$

$$v(m_1 - 1) = v(2,4; 2,3) = \frac{4}{21} \approx 0.190476.$$

Матрица очередного хода:

	<i>man</i>	<i>mouse</i>
<i>cat</i>	$\frac{2}{7}$	$\frac{16}{21}$
<i>woman</i>	$\frac{2}{3}$	$\frac{4}{21}$

Нижняя цена игры:

$$\underline{v} = \frac{2}{7} \approx 0.285714.$$

Верхняя цена игры:

$$\overline{v} = \frac{2}{3} \approx 0.666667.$$

Седловой точки нет, поэтому игра решается в смешанных стратегиях.

Цена игры:

$$v(3,4; 2,3) = \frac{10}{21} \approx 0.47619.$$

Оптимальная стратегия группы I:

$$x = (cat, woman) = \left(\frac{1}{2}, \frac{1}{2}\right).$$

Оптимальная стратегия группы II:

$$y = (man, mouse) = \left(\frac{3}{5}, \frac{2}{5}\right).$$

Проверим значение по рекуррентной формуле:

$$v = \frac{v(m_1 - 1)v(m_2 - 1) - v(n_1 - 1)v(n_2 - 1)}{v(m_1 - 1) + v(m_2 - 1) - v(n_1 - 1) - v(n_2 - 1)}.$$

Подстановка вычисленных значений даёт:

$$v = \frac{10}{21}.$$

Значение совпадает с ценой матричной игры.

Вывод:

В работе были рассмотрены игры на разорение на примере задачи «инспектирование» и задачи «женщины и кошки против мышей и мужчин».

Для игры «инспектирование»:

$$N = 13$$

получена цена игры:

$$v_{13} = \frac{6}{7} \approx 0.857143.$$

Оптимальные стратегии обоих игроков совпадают:

$$x^{(13)} = y^{(13)} = \left(\frac{1}{14}, \frac{13}{14} \right).$$

Для игры «женщины и кошки против мышей и мужчин» при параметрах:

$$m_1 = 3, m_2 = 4, n_1 = 2, n_2 = 3$$

получена цена игры:

$$v(3,4; 2,3) = \frac{10}{21} \approx 0.47619.$$

Оптимальная стратегия группы I:

$$(cat, woman) = \left(\frac{1}{2}, \frac{1}{2} \right).$$

Оптимальная стратегия группы II:

$$(man, mouse) = \left(\frac{3}{5}, \frac{2}{5} \right).$$

Таким образом, обе индивидуальные задачи решены в смешанных стратегиях, поскольку в соответствующих матрицах нижняя и верхняя цены игры не совпадают, то есть седловая точка в чистых стратегиях отсутствует.

Приложение (исходный код)

```
from __future__ import annotations

from dataclasses import dataclass
from functools import lru_cache
from fractions import Fraction
from typing import Iterable, Sequence

def F(x: int | float | Fraction) -> Fraction:
    if isinstance(x, Fraction):
        return x
    return Fraction(x).limit_denominator()

def fmt_frac(x: Fraction | int | float, show_decimal: bool = True, digits: int = 6) -> str:
    value = F(x)
    if value.denominator == 1:
        base = str(value.numerator)
    else:
        base = f"{value.numerator}/{value.denominator}"

    if not show_decimal:
        return base

    dec = float(value)
    if abs(dec - round(dec)) < 10 ** (-(digits - 1)):
        return base
    return f"{base} ≈ {dec:.{digits}f}".rstrip("0").rstrip(".")

def fmt_strategy(strategy: tuple[Fraction, Fraction], labels: tuple[str, str]) -> str:
    return (
        f"({labels[0]}: {fmt_frac(strategy[0])}); "
        f"{labels[1]}: {fmt_frac(strategy[1])})"
    )

def print_matrix_2x2(matrix: tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]],
                    row_labels: tuple[str, str] = ("1", "2"),
                    col_labels: tuple[str, str] = ("1", "2")) -> None:
    width = 18
    print(f"{'':>12}{col_labels[0]:>{width}}{col_labels[1]:>{width}}")
    for label, row in zip(row_labels, matrix):
        print(f"{label:>12}{fmt_frac(row[0]):>{width}}{fmt_frac(row[1]):>{width}}")

@dataclass(frozen=True)
class Matrix2x2Solution:
    matrix: tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]]
    value: Fraction
    row_strategy: tuple[Fraction, Fraction]
    col_strategy: tuple[Fraction, Fraction]
    lower_price: Fraction
    upper_price: Fraction
    saddle_points: tuple[tuple[int, int], ...]
    method: str

def solve_zero_sum_2x2(matrix_like: Sequence[Sequence[int | float | Fraction]]) -> Matrix2x2Solution:
    if len(matrix_like) != 2 or any(len(row) != 2 for row in matrix_like):
        raise ValueError("Ожидается матрица размера 2x2")

    matrix = tuple(tuple(F(x) for x in row) for row in matrix_like) # type: ignore[assignment]
    a, b = matrix[0]
    c, d = matrix[1]

    row_mins = (min(matrix[0]), min(matrix[1]))
    lower_price = max(row_mins)
    col_maxs = (max(matrix[0][0], matrix[1][0]), max(matrix[0][1], matrix[1][1]))
    upper_price = min(col_maxs)

    saddle_points: list[tuple[int, int]] = []
    if lower_price == upper_price:
        for i in range(2):
            for j in range(2):
                if matrix[i][j] == lower_price and matrix[i][j] == row_mins[i] and matrix[i][j] == col_maxs[j]:
                    saddle_points.append((i, j))
    if saddle_points:
        i, j = saddle_points[0]
        row_strategy = (Fraction(1), Fraction(0)) if i == 0 else (Fraction(0), Fraction(1))
```

```

col_strategy = (Fraction(1), Fraction(0)) if j == 0 else (Fraction(0), Fraction(1))
return Matrix2x2Solution(
    matrix=matrix,
    value=lower_price,
    row_strategy=row_strategy,
    col_strategy=col_strategy,
    lower_price=lower_price,
    upper_price=upper_price,
    saddle_points=tuple(saddle_points),
    method="седловая точка, чистые стратегии",
)

denominator = a - b - c + d
if denominator == 0:
    raise ValueError(
        "Не удалось найти внутреннее смешанное решение: знаменатель равен 0, "
        "а седловая точка отсутствует. Проверьте матрицу."
    )

p_row1 = (d - c) / denominator
q_col1 = (d - b) / denominator
value = (a * d - b * c) / denominator

row_strategy = (p_row1, 1 - p_row1)
col_strategy = (q_col1, 1 - q_col1)

if any(x < 0 or x > 1 for x in row_strategy + col_strategy):
    raise ValueError(
        "Получились вероятности вне [0,1]. Для данной матрицы требуется "
        "дополнительный анализ доминирования/вырождения."
    )

return Matrix2x2Solution(
    matrix=matrix,
    value=value,
    row_strategy=row_strategy,
    col_strategy=col_strategy,
    lower_price=lower_price,
    upper_price=upper_price,
    saddle_points=tuple(),
    method="смешанные стратегии",
)

def print_matrix_solution(
    title: str,
    matrix: tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]],
    row_labels: tuple[str, str] = ("строка 1", "строка 2"),
    col_labels: tuple[str, str] = ("столбец 1", "столбец 2"),
) -> Matrix2x2Solution:
    print("\n" + title)
    print_matrix_2x2(matrix, row_labels=row_labels, col_labels=col_labels)
    solution = solve_zero_sum_2x2(matrix)
    print(f"Метод решения: {solution.method}")
    print(f"Нижняя цена игры: {fmt_frac(solution.lower_price)}")
    print(f"Верхняя цена игры: {fmt_frac(solution.upper_price)}")
    print(f"Цена игры: {fmt_frac(solution.value)}")
    print(f"Оптимальная стратегия игрока-строки: {fmt_strategy(solution.row_strategy, row_labels)}")
    print(f"Оптимальная стратегия игрока-столбца: {fmt_strategy(solution.col_strategy, col_labels)}")
    if solution.saddle_points:
        points = ", ".join(f"({i + 1}, {j + 1})" for i, j in solution.saddle_points)
        print(f"Седловые точки: {points}")
    return solution

def inspection_value_recursive(N: int) -> Fraction:
    if N < 1:
        raise ValueError("N должно быть не меньше 1")
    value = Fraction(0)
    for _ in range(2, N + 1):
        value = (value + 1) / (-value + 3)
    return value

def inspection_value_closed(N: int) -> Fraction:
    if N < 1:
        raise ValueError("N должно быть не меньше 1")
    return Fraction(N - 1, N + 1)

```



```

def inspection_matrix(N: int) -> tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]]:
    if N < 2:
        raise ValueError("Матрица  $\Gamma_N$  в виде 2x2 строится для  $N \geq 2$ ")
    previous = inspection_value_recursive(N - 1)
    return ((Fraction(-1), Fraction(1)), (Fraction(1), previous))

def inspection_matrix_closed(N: int) -> tuple[tuple[Fraction, Fraction], tuple[Fraction, Fraction]]:
    if N < 2:
        raise ValueError("Матрица  $\Gamma_N$  в виде 2x2 строится для  $N \geq 2$ ")
    return ((Fraction(-1), Fraction(1)), (Fraction(1), Fraction(N - 2, N)))

def inspection_optimal_strategy(N: int) -> tuple[Fraction, Fraction]:
    if N < 2:
        raise ValueError("Оптимальная смешанная стратегия в указанном виде дана для  $N \geq 2$ ")
    return Fraction(1, N + 1), Fraction(N, N + 1)

def print_inspection_lecture_check(max_N: int = 8) -> None:
    print("\n" + "=" * 96)
    print("Лекционный пример 2: инспектирование")
    print("=" * 96)
    print("Рекурсия:  $v_1 = 0, \quad v_N = (v_{N-1}+1)/(-v_{N-1}+3)$ ")
    print("Замкнутая формула:  $v_N = (N-1)/(N+1)$ ")
    print("Оптимальные стратегии при  $N \geq 2$ :  $x^N = y^N = (1/(N+1), N/(N+1))$ ")

    print("\nПроверочная таблица:")
    print(f"{'N':>3} {'v_N рекурсивно':>22} {'v_N формула':>22} {'цена матрицы  $\Gamma_N$ ':>22} {'x=y':>32}")
    for N in range(1, max_N + 1):
        v_rec = inspection_value_recursive(N)
        v_closed = inspection_value_closed(N)
        if N == 1:
            matrix_value = "-"
            strategy = "-"
        else:
            sol = solve_zero_sum_2x2(inspection_matrix(N))
            matrix_value = fmt_frac(sol.value)
            strategy = f"({fmt_frac(sol.row_strategy[0], False)}, {fmt_frac(sol.row_strategy[1], False)})"
        print(
            f"{'N':>3} {fmt_frac(v_rec):>22} {fmt_frac(v_closed):>22} "
            f"{matrix_value:>22} {strategy:>32}"
        )

def print_inspection_individual_variant(N: int) -> None:
    print("\n" + "=" * 96)
    print(f"Индивидуальный вариант: инспектирование,  $N = \{N\}$ ")
    print("=" * 96)
    if N < 2:
        raise ValueError("Для индивидуального варианта используйте  $N \geq 2$ ")

    print("Последовательный расчет по рекурсии:")
    previous = Fraction(0)
    print(f"  $v_1 = \{fmt\_frac(previous)\}$ ")
    for k in range(2, N + 1):
        current = (previous + 1) / (-previous + 3)
        print(
            f"  $v_{\{k\}} = (v_{\{k-1\}}+1)/(-v_{\{k-1\}}+3) = "$ 
            f" $\{fmt\_frac(previous, False)\}+1)/(-\{fmt\_frac(previous, False)\}+3) = "$ 
            f" $\{fmt\_frac(current)\}$ "
        )
        previous = current

    v_rec = inspection_value_recursive(N)
    v_closed = inspection_value_closed(N)
    x = inspection_optimal_strategy(N)
    y = inspection_optimal_strategy(N)

    print("\nПроверка по замкнутой формуле:")
    print(f"  $v_{\{N\}} = (N-1)/(N+1) = \{N-1\}/\{N+1\} = \{fmt\_frac(v\_closed)\}$ ")
    print(f" Значение по рекурсии:  $\{fmt\_frac(v\_rec)\}$ ")
    print(f" Совпадение рекурсии и формулы: {'да' if v_rec == v_closed else 'нет'}")

    matrix = inspection_matrix(N)
    print_matrix_solution(
        title=f"Эффективная матрица  $\Gamma_{\{N\}} = [[-1, 1], [1, v_{\{N-1\}}]]$ ",
        matrix=matrix,
        row_labels=("I", "II"),
        col_labels=("I", "II"),
    )

```

```

)

print("\nИтоговые значения для отчета:")
print(f" Цена игры:  $v_{\{N\}} = \{fmt\_frac(v\_rec)\}$ ")
print(f" Оптимальная стратегия первого игрока:  $x^{\{N\}} = \{fmt\_strategy(x, ('I', 'II'))\}$ ")
print(f" Оптимальная стратегия второго игрока:  $y^{\{N\}} = \{fmt\_strategy(y, ('I', 'II'))\}$ ")

@dataclass(frozen=True)
class WomenCatsMiceMenGame:
    @staticmethod
    def _validate_state(m1: int, m2: int, n1: int, n2: int) -> None:
        if min(m1, m2, n1, n2) < 0:
            raise ValueError("Количество участников не может быть отрицательным")

    @lru_cache(maxsize=None)
    def value(self, m1: int, m2: int, n1: int, n2: int) -> Fraction:
        self._validate_state(m1, m2, n1, n2)

        if (n1 == 0 or n2 == 0) and m1 > 0 and m2 > 0:
            return Fraction(1)

        if (m1 == 0 or m2 == 0) and n1 > 0 and n2 > 0:
            return Fraction(-1)

        if m1 == 0 or m2 == 0 or n1 == 0 or n2 == 0:
            return Fraction(0)

        matrix = self.subgame_matrix(m1, m2, n1, n2)
        return solve_zero_sum_2x2(matrix).value

    def subgame_matrix(self, m1: int, m2: int, n1: int, n2: int) -> tuple[
        tuple[Fraction, Fraction], tuple[Fraction, Fraction]]:
        self._validate_state(m1, m2, n1, n2)
        if min(m1, m2, n1, n2) <= 0:
            raise ValueError("Матрица очередного хода строится только для  $m1, m2, n1, n2 > 0$ ")

        return (
            (
                self.value(m1, m2 - 1, n1, n2), # cat vs man: man eliminates cat
                self.value(m1, m2, n1 - 1, n2), # cat vs mouse: cat eliminates mouse
            ),
            (
                self.value(m1, m2, n1, n2 - 1), # woman vs man: woman eliminates man
                self.value(m1 - 1, m2, n1, n2), # woman vs mouse: mouse eliminates woman
            ),
        )

    def solve_state(self, m1: int, m2: int, n1: int, n2: int) -> Matrix2x2Solution:
        return solve_zero_sum_2x2(self.subgame_matrix(m1, m2, n1, n2))

    def recurrence_formula_value(self, m1: int, m2: int, n1: int, n2: int) -> Fraction:
        a = self.value(m1, m2 - 1, n1, n2) #  $v(m2-1)$ 
        b = self.value(m1, m2, n1 - 1, n2) #  $v(n1-1)$ 
        c = self.value(m1, m2, n1, n2 - 1) #  $v(n2-1)$ 
        d = self.value(m1 - 1, m2, n1, n2) #  $v(m1-1)$ 
        denominator = a + d - b - c
        if denominator == 0:
            raise ZeroDivisionError("Знаменатель рекуррентной формулы равен 0")
        return (a * d - b * c) / denominator

    def generated_population_variant(variant: int) -> tuple[int, int, int, int]:
        m1 = 2 + (variant % 3)
        m2 = 2 + ((variant + 1) % 3)
        n1 = 2 + ((variant + 2) % 3)
        n2 = 2 + ((variant + 3) % 3)
        return m1, m2, n1, n2

    def print_women_cats_lecture_check() -> None:
        print("\n" + "=" * 96)
        print("Лекционный пример 3: женщины и кошки против мышей и мужчин")
        print("=" * 96)
        print("Граничные условия:")
        print("  $v(m1, m2; n1, 0) = v(m1, m2; 0, n2) = 1$ , если  $m1, m2 > 0$ ")
        print("  $v(m1, 0; n1, n2) = v(0, m2; n1, n2) = -1$ , если  $n1, n2 > 0$ ")
        print("Проверяется значение  $v(1, 1; 1, 1)$ , которое по лекции равно 0.")

        game = WomenCatsMiceMenGame()
        params = (1, 1, 1, 1)
        matrix = game.subgame_matrix(*params)

```

```

solution = print_matrix_solution(
    title="Матрица для состояния v(1,1;1,1)",
    matrix=matrix,
    row_labels=("cat", "woman"),
    col_labels=("man", "mouse"),
)
print(f"Итого: v(1,1;1,1) = {fmt_frac(solution.value)}")

def print_women_cats_individual_variant(m1: int, m2: int, n1: int, n2: int) -> None:
    print("\n" + "=" * 96)
    print(f"Индивидуальный вариант: v(m1,m2;n1,n2) = v({m1},{m2};{n1},{n2})")
    print("=" * 96)

    game = WomenCatsMiceMenGame()
    matrix = game.subgame_matrix(m1, m2, n1, n2)

    print("Смысл параметров:")
    print(f"  Группа I: m1={m1} женщин, m2={m2} кошек")
    print(f"  Группа II: n1={n1} мышей, n2={n2} мужчин")

    print("\nЗначения подыгр, входящих в рекуррентную матрицу:")
    print(f"  v(m2-1) = v({m1},{m2-1};{n1},{n2}) = {fmt_frac(matrix[0][0])}")
    print(f"  v(n1-1) = v({m1},{m2};{n1-1},{n2}) = {fmt_frac(matrix[0][1])}")
    print(f"  v(n2-1) = v({m1},{m2};{n1},{n2-1}) = {fmt_frac(matrix[1][0])}")
    print(f"  v(m1-1) = v({m1-1},{m2};{n1},{n2}) = {fmt_frac(matrix[1][1])}")

    solution = print_matrix_solution(
        title="Матрица очередного хода индивидуального варианта",
        matrix=matrix,
        row_labels=("cat", "woman"),
        col_labels=("man", "mouse"),
    )

    formula_value = game.recurrence_formula_value(m1, m2, n1, n2)
    print("\nПроверка по рекуррентной формуле:")
    print(f"  v = (v(m1-1)*v(m2-1) - v(n1-1)*v(n2-1)) /")
    print(f"        (v(m1-1)+v(m2-1)-v(n1-1)-v(n2-1))")
    print(f"  Значение по формуле: {fmt_frac(formula_value)}")
    print(f"  Совпадение с ценой матричной игры: {'да' if formula_value == solution.value else 'нет'}")

    print("\nИтоговые значения для отчета:")
    print(f"  Цена игры: v({m1},{m2};{n1},{n2}) = {fmt_frac(solution.value)}")
    print(
        "  Оптимальная стратегия группы I: "
        + fmt_strategy(solution.row_strategy, ("cat", "woman"))
    )
    print(
        "  Оптимальная стратегия группы II: "
        + fmt_strategy(solution.col_strategy, ("man", "mouse"))
    )
    print(f"  Количество вычисленных рекурсивных состояний: {game.value.cache_info().currsz}")

VARIANT = 13
INSPECTION_N = VARIANT
POPULATION_PARAMS = generated_population_variant(VARIANT)

def solve_ruin_games() -> None:
    print("Лабораторная работа: Игры на разорение")
    print(f"Вариант: {VARIANT}")
    print(f"Индивидуальный N для инспектирования: {INSPECTION_N}")
    print(
        "Индивидуальные параметры для игры 'женщины и кошки против мышей и мужчин': "
        f"m1={POPULATION_PARAMS[0]}, m2={POPULATION_PARAMS[1]}, "
        f"n1={POPULATION_PARAMS[2]}, n2={POPULATION_PARAMS[3]}"
    )

    print_inspection_lecture_check(max_N=8)
    print_inspection_individual_variant(N=INSPECTION_N)

    print_women_cats_lecture_check()
    print_women_cats_individual_variant(*POPULATION_PARAMS)

if __name__ == "__main__":
    solve_ruin_games()

```